

TEST AUTOMATION FRAMEWORK PROJECT REPORT

Capstone Assignment 1: Automating an E-Commerce Web Application
Using Selenium WebDriver with Python

Application Under Test	automationexercise.com
Prepared By	Dipanjan Sheth
Technology Stack	Python • Selenium WebDriver • Pytest
Date	24 September 2026

Selenium WebDriver • Python • Pytest • Page Object Model

TEST AUTOMATION FRAMEWORK REPORT

Capstone Assignment 1: Automating an E-Commerce Web Application Using Selenium WebDriver with Python

Application Under Test: automationexercise.com

Prepared By: Dipanjan Sheth

Technology Stack: Python, Selenium WebDriver, Pytest

Date: 24 September 2026

1. Introduction

This report documents the design, implementation, and execution of an automated test framework built to validate an end-to-end online shopping flow. Developed for Capstone Assignment 1, the framework uses Selenium WebDriver with Python to automate a purchase journey on a public e-commerce platform. The suite covers core capabilities including authentication, product search, cart manipulation, modal/alert handling, data-driven test execution, per-step visual logging, and self-contained HTML report generation.

The target application, automationexercise.com, serves as a demo e-commerce web application designed for test automation practice. It includes interactive elements suitable for testing all ten assignment requirements.

2. Objectives

The main objective of this project is to build a reliable, maintainable, and data-driven Selenium test framework that simulates a customer's shopping journey and verifies system state at each step. Key goals include:

- **Controlled Session Management:** Launch and manage browser instances reproducibly.
- **Authentication:** Authenticate users, dynamically handling existing accounts and new user registration.
- **Product Search & Catalog Navigation:** Search the catalog and verify search results.
- **Cart Management:** Adjust product quantity controls and update cart state accurately.
- **Data Verification:** Validate line items, quantities, and properties on the cart page.
- **Visual Audit Trail:** Capture step-by-step PNG screenshots during test execution.
- **Data-Driven Execution:** Externalize test inputs into JSON and Excel files.
- **Pop-up & Modal Interruption Handling:** Automatically handle JavaScript browser alerts and DOM Bootstrap modals.
- **Reporting:** Generate self-contained HTML test execution reports with embedded media.

3. Scope

The scope encompasses an end-to-end purchasing workflow: user login/signup, product search, quantity adjustment on the product detail page, cart item addition, and cart content verification. Cross-browser capability (Google Chrome and Mozilla Firefox) and execution modes (headed and headless) are fully supported.

4. Tools and Technologies Used

Category	Tool / Technology
Programming Language	Python 3.12
Automation Library	Selenium WebDriver 4.21 (Selenium Manager for automatic driver binaries)
Test Framework	pytest 8.x

Category	Tool / Technology
Reporting Engine	pytest -html 4.1 (self-contained HTML output)
Data Handling	openpyxl (Excel processing), built-in json module
Design Architecture	Page Object Model (POM) Pattern
Application Under Test	https://automationexercise.com/ (E-Commerce Practice Portal)
Supported Browsers	Google Chrome (Default), Mozilla Firefox (Headed & Headless)

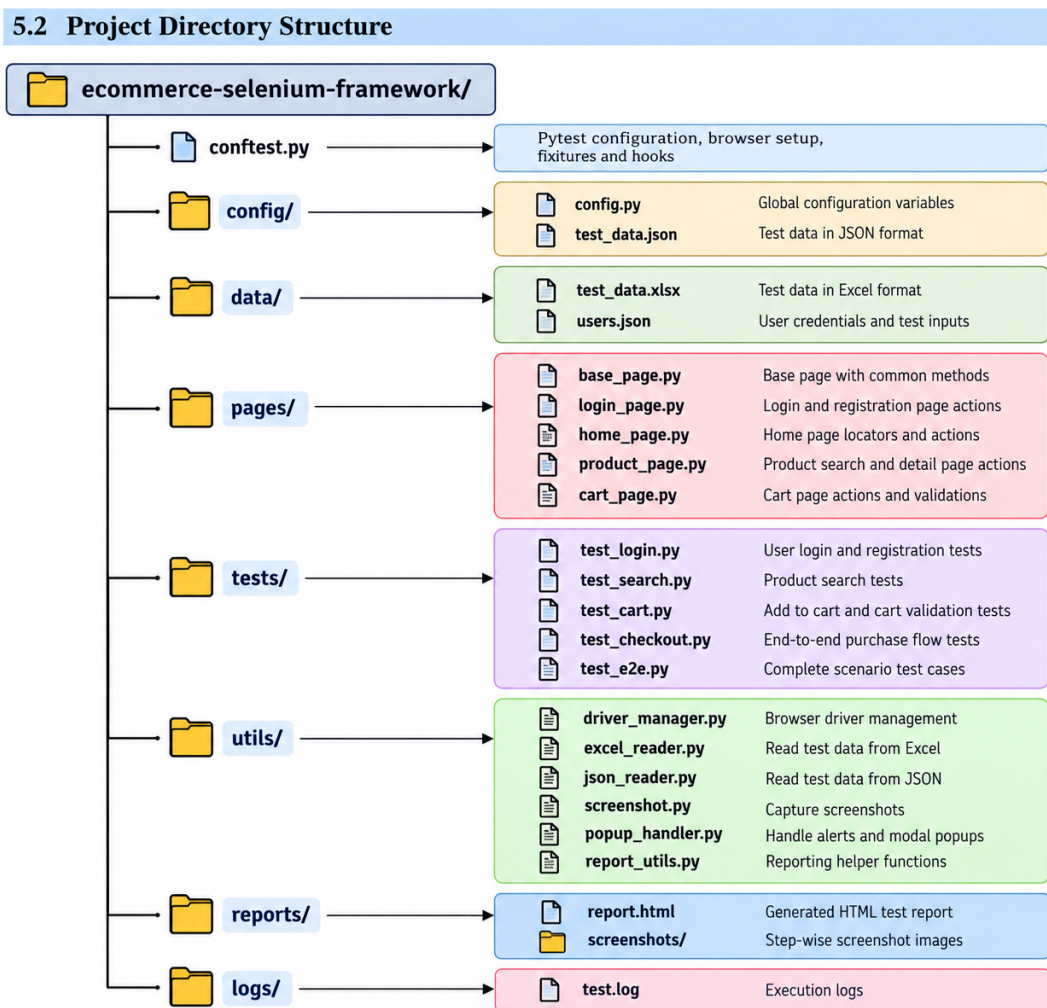
5. System & Framework Architecture

The framework follows the Page Object Model (POM) design pattern, which separates web page representation (locators and actions) from test logic (business steps and assertions) to maximize maintainability.

5.1 Layered Structure Architecture

TEST LAYER (tests/)	Defines test cases, business flows, and assertions.
PAGE OBJECT LAYER (pages/)	Encapsulates UI locators and page interaction logic.
UTILITY LAYER (utils/)	Provides driver management, data parsing, screenshot capture, and popup handling.
CONFIGURATION LAYER	Uses conftest.py and pytest.ini for fixtures, command-line options, and reporting hooks.

5.2 Project Directory Structure



6. Requirement-to-Implementation Mapping

#	Requirement	Implementation	Technical Details
1	Launch browser	utils/driver_factory.py	Selenium 4 Chrome/Firefox launcher with Selenium Manager auto driver resolution.
2	Login / Authentication	pages/login_page.py	LoginPage attempts login; SignupFormPage auto-registers a new account if needed.
3	Search product	pages/products_page.py	Executes search_product() and verifies results via has_search_results().
4	Add product to cart	pages/products_page.py	Handles interaction via ProductDetailsPage.add_to_cart().
5	Update quantity	pages/products_page.py	Modifies product count using ProductDetailsPage.set_quantity().
6	Verify cart details	pages/cart_page.py	Extracts and asserts items using CartPage.get_cart_line_items().
7	Capture screenshots	utils/screenshot_util.py	Timestamped PNG capture invoked at every step and linked in pytest hooks.
8	External Test Data	utils/data_reader.py	Unified parser reading from test_data.json and test_data.xlsx.
9	Handle popup/alerts	utils/popup_handler.py	Manages native JS alerts (switch_to.alert) and Bootstrap UI modal popups.
10	Execution report	pytest.ini / pytest-html	Generates report/execution_report.html with embedded step screenshots.

7. Test Scenario Walkthrough

The core execution scenario in tests/test_purchase_flow.py runs through a 10-step sequence:

- 1. Browser Launch:** Initializes the WebDriver session and navigates to the application URL.
- 2. Authentication Check:** Navigates to the login screen and submits credentials from test data.
- 3. Self-Healing Signup Fallback:** Automatically triggers account creation if credentials are not recognized.
- 4. Catalog Search:** Enters the search term (e.g., "Dress") on the Products page.
- 5. Result Verification:** Confirms the "Searched Products" section displays matching items.
- 6. Product View:** Selects the target item to open its Product Detail page.
- 7. Quantity Adjustment:** Sets the specified quantity parameter (e.g., 3 units) before adding to cart.
- 8. Cart Insertion & Modal Dismissal:** Adds product and dismisses the DOM-based "Added!" modal.
- 9. Cart Navigation:** Opens the shopping cart and reads back active line items.
- 10. Data Assertion:** Validates product name, unit price, total price, and correct quantity allocation.

8. Test Data Management

Test inputs are decoupled from test scripts to enable data-driven capabilities:

- **JSON Source:** test_data/test_data.json houses core environment details and user profiles.
- **Excel Source:** test_data/test_data.xlsx provides equivalent multi-sheet data (General, User, Signup).
- **Abstraction API:** The function load_test_data(source="json"|"excel") standardizes data structures, allowing seamless switching without test script edits.

9. Popup and Interruption Handling

The framework implements two specialized utilities in `utils/popup_handler.py`:

- 1. Native JavaScript Dialogs:** Handled via `handle_js_alert_if_present()` using standard Selenium driver controls (`driver.switch_to.alert`) with conditional try-except blocks.
- 2. DOM-based Bootstrap Overlays:** Handled via `close_continue_shopping_modal_if_present()` using `WebDriverWait` to locate and click the "Continue Shopping" button safely without throwing wait timeouts.

10. Execution Reporting & Screenshot Capture

Visual capturing occurs at every operational checkpoint, generating labeled PNG files in `screenshots/`. The `pytest_runtest_makereport` hook in `conf/test.py` automatically captures the final execution state and embeds base64 screenshot data directly into `report/execution_report.html`. This yields a single standalone HTML document that can be distributed without external image assets.

11. Suite Execution Guide

Execute the framework using standard command-line interfaces from the project root:

```
\# 1. Install dependencies pip install -r requirements.txt # 2. Generate optional Excel data source python test_data/generate_excel_data.py # 3. Default Execution (Chrome, Headed mode) pytest # 4. Customized Execution (Firefox, Headless mode) pytest --browser=firefox --headless
```

12. Challenges & Design Decisions

Challenge Identified	Framework Mitigation & Design Decision
Ephemeral Test Accounts	The AUT resets user databases periodically. Implemented dynamic registration logic that falls back to signing up a user if login fails on a fresh run.
Read-Only Cart Quantities	The cart table disables quantity input. The framework configures target quantities on the Product Detail Page prior to cart addition.
Non-Native UI Overlays	Modal overlays block subsequent UI clicks. Explicit wait utilities handle DOM overlays distinctly from native browser alerts.

13. Conclusion

This project delivers a complete, maintainable, and robust Selenium WebDriver automation framework using Python and pytest. By combining the Page Object Model, externalized data parsing, dual-layer popup handling, step-by-step visual audits, and embedded HTML reporting, the framework fully meets all ten capstone project requirements.